
Evaluating Temporal Semantic Caching and Workflow Optimization in Agentic Plan-Execute Pipelines

Alimurtaza Merchant*
Columbia University
amm2640@columbia.edu

Krish Veera*
Columbia University
krv2123@columbia.edu

Sajal Kumar Goyla*
Columbia University
sg4607@columbia.edu

Shambhawi Bhure*
Columbia University
sb5185@columbia.edu

Dhaval Patel
IBM
pateldha@us.ibm.com

Kaoutar El Maghraoui
IBM Research
kelmaghr@us.ibm.com

Abstract

Industrial asset operations workflows are latency-sensitive because a single user query may require coordination over sensor data, work orders, failure modes, forecasting tools, and domain-specific agents. We evaluate this problem on AssetOpsBench (AOB), an industrial agent benchmark whose plan-execute pipeline exposes repeated overhead from tool discovery, LLM planning, MCP tool execution, and final summarization. Existing LLM caching techniques such as KV-cache reuse and embedding-based semantic caching were designed for chatbot serving and break down when output validity depends on time, asset, or sensor parameters. We propose two complementary optimization layers for AOB plan-execute pipelines: a temporal semantic cache and a set of MCP workflow optimizations combining disk-backed tool-discovery caching and dependency-aware parallel step execution. MCP workflow optimizations corresponded to a $1.67\times$ speedup and reduced median end-to-end latency by about 40.0% while the temporal-cache benchmark achieved a median of $30.6\times$ speedup on cache hits. Beyond the speedup, our results expose a concrete failure mode of pure semantic caching for parameter-rich industrial queries, providing a critical analysis of how caching choices interact with evaluation correctness in MCP-backed agent benchmarks.

1 Introduction

LLM-based agents increasingly serve as orchestration layers over domain-specific tools and data sources (1; 2; 3). Many such agents follow the Plan-Execute paradigm: a planner LLM decomposes a user query into a sequence of tool calls, and an executor invokes those tools, often through standardized interfaces such as the Model Context Protocol (MCP) (4). While expressive, this two-stage structure introduces substantial wall-clock latency. A single query can require tool discovery, multi-step planning, multiple MCP tool invocations, and a final summarization pass before an answer is returned.

This latency is particularly acute in industrial asset operations, where queries naturally span heterogeneous data sources: sensor telemetry, work orders, failure modes, and time-series forecasts. The AssetOpsBench benchmark (5) formalizes this setting with four MCP-backed domain servers

*Equal contribution.

and a corpus of human-authored operational queries. In practice, the same operator may issue many semantically related queries against the same assets: paraphrases, repetitions, parameter shifts (Chiller 6 vs. Chiller 9), or time-window shifts (yesterday vs. last week). A naive plan-execute implementation pays the full orchestration cost on every query, and at paraphrase scale this makes systematic evaluation of MCP-backed agents prohibitively slow.

Caching is the standard remedy for repeated computation in LLM serving, but existing techniques were designed for chatbot workloads. Context (KV) caching (6; 7; 8; 9) reuses prefill states for identical prefixes; semantic caching (10; 11) reuses (input, output) pairs across paraphrases via embedding similarity. Neither approach matches the structure of industrial agent queries, where output validity depends on external state (asset, sensor, time window) that is not visible in the query embedding. Recent work on Agentic Plan Caching (12) addresses part of this gap by caching plan templates rather than answers, but does not address temporal validity, which is central in industrial telemetry settings where “what happened yesterday” resolves to a different window each day.

Our approach. We propose two optimization layers for AOB plan-execute pipelines. At the query level, we build a *temporal semantic cache* that classifies each query into Volatile (live state, never cached), Anchored (time-bounded, matched against compatible windows), or Static (no temporal dependence, standard semantic match) before invoking embedding-based retrieval and a reranker-based judge. At the workflow level, we add two *MCP optimizations*: disk-backed tool-discovery caching and dependency-aware parallel step execution over a persistent server pool. The two layers are independently beneficial and additive: the MCP layer reduces latency on every query regardless of cache state, while the cache layer adds large further savings when a query resolves to a valid hit.

Contributions.

1. **Temporal semantic caching for industrial agents.** We extend semantic caching with a pre-retrieval temporal classifier and a window-aware judge that addresses the parameter-and-time sensitivity of industrial queries.
2. **MCP workflow optimizations.** We combine discovery-phase caching, DAG-layered parallel execution, and a persistent server pool to reduce per-query orchestration overhead in MCP-backed plan-execute pipelines.
3. **Refined evaluation setup for AOB.** We provide a paired baseline-vs-optimized harness, a paraphrase-tier test set with parent-id-based ground truth for cache hit/miss labelling, and per-phase latency profiling that makes systematic AOB ablations tractable on a single machine.
4. **Critical analysis of caching as an evaluation choice.** We expose a concrete failure mode: pure semantic similarity is not a sound proxy for answer validity in parameter-rich AOB queries, capping hit-decision F1 near 0.67 in our setting. This gives a measurable handle on when caching is safe to use as part of an evaluation pipeline, and when it is not.

2 Background and Motivation

2.1 AssetOpsBench and MCP-backed Plan-Execute Agents

AssetOpsBench (AOB) (5) is a benchmark for evaluating LLM agents on industrial asset operations and maintenance workflows. AOB exposes four specialized domain servers covering Internet of Things (IoT) telemetry, Failure Mode and Sensor Relation (FMSR), Time Series Foundation Models (TSFM) (13), and Work Order (WO) records, all wrapped under the Model Context Protocol (MCP) (4). The benchmark scenarios are written as human-authored natural-language operational queries rather than database-level API calls, reflecting the language an operator or reliability engineer would actually use.

In a Plan-Execute pipeline (1), a query is not answered by a single LLM call. Instead, the workflow decomposes into four phases: *Discovery*, which spawns MCP servers and collects tool signatures via `list_tools()`; *Planning*, which uses an LLM to convert the query and tool catalog into a structured plan; *Execution*, which resolves tool arguments and invokes the relevant MCP tools; and *Summarization*, which uses an LLM to synthesize the tool outputs into a response. Figure 1 illustrates this structure and contrasts the baseline path with our optimized path.

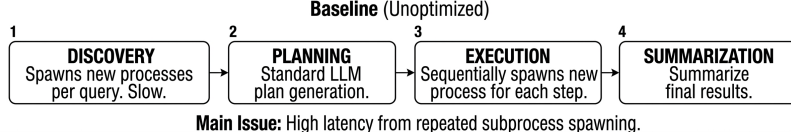


Figure 1: MCP Workflow.

The Plan-Execute abstraction is useful because it exposes a structured plan before tool execution begins. However, this separation does not automatically imply parallelism: many implementations consume the generated plan strictly sequentially. The optimization opportunity comes from treating the plan as a directed acyclic graph and dispatching dependency-independent steps concurrently, while preserving order across true dependencies.

2.2 LLM Caching for Agents: Methods and Limitations

Caching is one of the most widely adopted techniques for reducing LLM serving cost. *Context caching* (6; 7; 8; 9) stores key-value pairs from the prefill phase and reuses them when prompt prefixes recur. *Semantic caching* (10; 11) stores (input, output) pairs and matches new queries by embedding similarity, exploiting the fact that paraphrases share underlying intent. *Plan caching* (12) caches plan templates extracted from completed agent executions and adapts them to new queries with a lightweight model. We find that all three families have limitations specific to MCP-backed industrial agent benchmarks like AOB:

1) Static-Output Assumption. Semantic caching assumes that outputs depend only on the input prompt (10; 11). This holds for chatbots but fails in AOB, where outputs depend on external state queried at run time. For example, “what is the status of work order WO-1234” returns a different answer depending on whether the order is open or closed, but the query text and its embedding are identical each time. A cache that keys on the input alone cannot detect that the stored answer is stale.

2) Parameter Insensitivity. Embedding-based retrieval captures the linguistic structure of a query but is insensitive to its operational parameters. “Failure modes detectable by Chiller 6 Efficiency sensor” and “Failure modes detectable by Chiller 9 Efficiency sensor” embed close together because they share the same sentence frame, yet they require disjoint tool calls and produce disjoint answers. Threshold tuning trades false positives against false negatives but does not eliminate this structural mismatch between what the embedding encodes and what determines a correct answer (11).

3) Temporal Blindness. Many AOB queries contain explicit or relative time expressions: “last week,” “yesterday,” “the past 24 hours.” Pure embedding similarity treats two such queries as equivalent regardless of their resolved windows, which is incorrect when the underlying telemetry has changed. Existing semantic caching frameworks (10; 11) do not expose a mechanism to distinguish queries that differ only in their resolved temporal anchor.

These limitations motivate a temporal-aware cache that distinguishes *semantic relatedness* from *safe answer reuse*, combined with workflow-level optimizations that reduce per-query overhead even on cache misses.

2.3 Related Work

Agent memory and plan reuse. Several recent systems augment LLM agents with external memory (14; 15; 16). Agent Workflow Memory (17) extracts and reuses workflow patterns to improve task success rates. Asteria (18) provides the semantic-cache primitives we build on (ANN over query embeddings, a reranker-based judger, LCFU eviction, and Markov prefetching) for general agentic LLM tool access. Agentic Plan Caching (12) extends agent-side reuse to a serving-cost objective by caching plan templates and adapting them with a small LM. Our work differs in two ways: we target MCP-backed industrial benchmarks where temporal validity is central, and we add a temporal classification layer in front of Asteria-style retrieval to handle relative time expressions and live-state queries.

LLM serving infrastructure. Modern LLM serving systems such as vLLM (19) and SGLang (20) optimize inference at the engine level through KV-cache management and structured generation. Our optimizations sit one layer up: they target the agent orchestration loop and are compatible with any underlying serving engine.

Multi-agent orchestration and benchmarks. Multi-agent collaboration has been studied in systems such as Mixture-of-Agents (21) and surveyed broadly in (22; 23). Agent benchmarks like GAIA (24) and Minions (25) evaluate task success and cost; AOB (5) extends this to industrial operations with MCP-exposed tooling. Our contribution is a refinement of the AOB evaluation setup itself, making at-scale paraphrase-tier ablations practical.

3 The Optimization Framework

3.1 Overview

Figure 2 shows the temporal semantic cache. Each incoming query is paired with a run-time timestamp and passed through a temporal classifier before semantic retrieval. The classifier assigns each query to one of four buckets. *Volatile* queries request live system state and bypass the cache. *Static* queries have no temporal dependence and enter semantic retrieval. *Relative* queries use expressions like “yesterday” or “last week” that are resolved against the run timestamp into concrete windows and then treated as Anchored. *Anchored queries* reference a fixed time window and enter approximate nearest-neighbor retrieval with a window-aware judger. On a hit, the cached answer is returned; on a miss, the query falls through to the full plan-execute pipeline, and the resulting answer is inserted into the cache. The MCP layer (Figure 1) sits inside this pipeline: even on a miss, discovery caching and parallel execution reduce wall-clock cost relative to the unoptimized baseline.

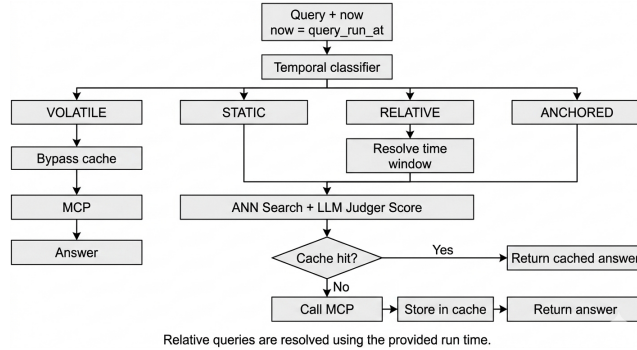


Figure 2: Temporal semantic cache workflow. A pre-retrieval temporal classifier routes each query: Volatile bypasses the cache; Static and resolved-Anchored queries enter ANN retrieval followed by a reranker-based judger.

3.2 Cache Layer Design Choices

Why temporal classification? A naive semantic cache would embed every query and search the index. This conflates linguistic similarity with answer reuse validity, as discussed in Section 2.2. Placing a temporal filter *before* retrieval lets us route each query into a regime where reuse is sound: live-state queries skip the cache entirely, time-bounded queries match only against compatible windows, and time-independent queries fall back to standard semantic retrieval. The classifier itself is a lightweight regex-based component that adds negligible per-query cost.

Anchored windows over relative phrases. Relative time expressions like “yesterday” resolve differently each day, so caching them under their literal text would produce stale hits. We resolve such phrases against the query timestamp at insertion time and store the concrete window with the cache entry. At lookup, the judger checks window compatibility as part of its acceptance decision.

Embedding plus reranker, not similarity alone. Cosine similarity over query embeddings is a coarse signal. We therefore use it only for candidate retrieval and route candidates through a

reranker-based judge that scores semantic and temporal alignment with the new query. This two-stage design (11) lets us tune retrieval recall and judging precision independently. Exact thresholds and model choices are listed in Appendix A.

3.3 MCP Workflow Optimizations

Discovery-phase caching. The baseline AOB pipeline performs MCP tool discovery on every query: spawning a Python subprocess for each of the four servers, establishing `stdio` connections, requesting the tool catalog via `list_tools()`, and terminating before planning begins. In our setup this consumes 2 to 3 seconds per query. We treat tool signatures as semi-static metadata and persist the aggregated catalog to a local JSON file. The cache key invalidates automatically on changes to server source code, server registrations, or project configuration; full key construction is in Appendix A.

Parallel step execution. We treat the generated plan as a directed acyclic graph of tool invocations and group steps into topological dependency layers. Independent steps within a layer execute concurrently, and dependency barriers preserve ordering across layers. To support concurrent execution, a persistent `MCPServerPool` maintains one `stdio` session per required server for the lifetime of a plan, with per-server asynchronous locks serializing concurrent calls to the same domain server while allowing inter-server concurrency. The executor is fail-tolerant: a failure on one MCP server does not halt sibling steps targeting other servers.

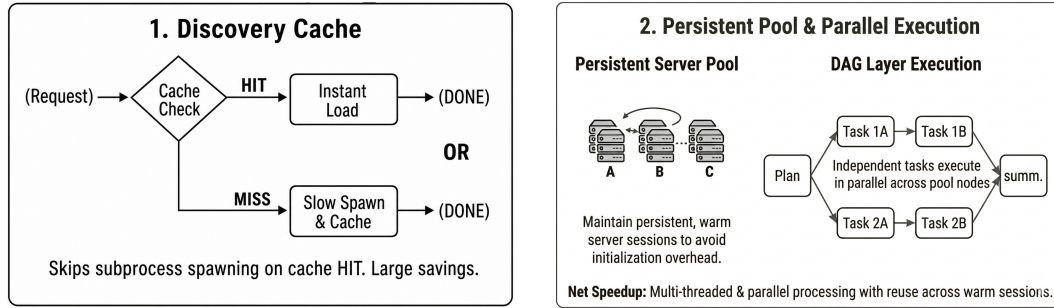


Figure 3: The optimized MCP Workflow component paths use a discovery cache and dispatch steps in parallel against a persistent pool.

4 Results and Evaluation

We evaluate the framework on AOB queries and report the following key findings:

- **End-to-end speedup.** The combined pipeline reduces median latency from 34.10s to 9.80s ($3.48\times$) on 80 paraphrase-tier queries (Section 4.3).
- **MCP workflow gains.** On 18 IoT queries, MCP optimizations alone yield a $1.67\times$ end-to-end speedup, with discovery cost reduced by $296\times$ and execution time by $1.99\times$ (Section 4.2).
- **Cache decision quality.** The temporal-classifier-plus-judge reaches F1 0.64 on hit/miss decisions in the combined system (Section 4.3), with the residual error concentrated on parameter-shifted queries.
- **Additive optimizations.** The miss path is faster than baseline because MCP gains apply regardless of cache state; the cache never makes performance worse.

4.1 Experiment Setup

Benchmark workload. All scenarios are drawn from `all_utterance.csv`, the hand-authored AOB corpus of 152 queries spanning IoT, FMSA, TSFM, Workorder, and multiagent types. Because the two optimization layers target different latency sources, we use two purposive subsets. For the *MCP-workflow scenarios*, we ran the AOB planner over the corpus and retained 20 queries whose

plans contained at least two parallelizable branches; the retained set covers all five query types. For the *cache scenarios*, we randomly partition parents into 20 *warm* parents and a held-out cold pool. We then use an LLM to generate semantically similar query paraphrases for each parent query, emit one paraphrase per warm parent as a 20-row seed CSV, and emit an 80-row test CSV split 60%/40% between warm-parent paraphrases (cache should hit) and cold-parent paraphrases (cache should miss). The `parent_id` membership in the warm set serves as ground truth for hit/miss labelling.

Baselines. For the workflow experiment, the baseline performs tool discovery on every query and executes plan steps sequentially. For the cache experiment, the baseline is the workflow-optimized pipeline with cache lookup and insert disabled, isolating the cache’s contribution from the workflow contribution. Each query is run paired under baseline and optimized conditions on the same simulated wall-clock so that row-level latency differences are attributable to the optimization rather than provider-side variance.

Metrics. We report per-query end-to-end latency with the median as primary statistic and the 5%-trimmed mean as a robustness check. Speedups are reported as the median of per-row ratios, robust to provider-side tail-latency events. For the workflow experiment, we additionally break out per-phase latency. For each test row the cache emits a *hit* or *miss*; pairing this with the `parent_id` ground truth produces a 2×2 confusion matrix from which we compute precision, recall, F_1 , and specificity. For misses we report median overhead (cached minus baseline latency).

Implementation. The plan-execute pipeline uses Llama-3.3-70B via LiteLLM for planning, tool-argument resolution, and summarization. The semantic cache uses Qwen3 embedding and reranker models. All experiments run on a single Apple M-series machine with 16 GB unified memory. Exact model strings, threshold values, cache capacity, and hardware are listed in Appendix A.

4.2 MCP Workflow Optimization Results

We evaluate the MCP layer in isolation on the 18 IoT queries from the AOB benchmark, each executed three times in baseline and optimized configurations (120 total profiled runs). Two queries (Q5, Q19) timed out across all attempts in both modes and are excluded.

Table 1: Aggregate phase-level comparison across 18 IoT queries (median of per-query medians, 3 runs each).

Phase	Baseline	Optimized	Saving	Speedup
Discovery	2.096s	0.007s	2.089s	296.08×
Planning (LLM)	10.285s	8.226s	2.059s	1.25×
Pre-fetch	0.415s	0.002s	0.413s	210.53×
Execution	34.639s	17.415s	17.224s	1.99×
Summarization	5.016s	5.165s	−0.149s	0.97×
Total	56.902s	34.164s	22.738s	1.67×

Phase-level effects are surgical. Discovery caching effectively eliminates per-query server-spawning overhead. Parallel execution with connection pooling reduces execution wall time by 1.99×. Planning and summarization, both dominated by LLM inference, show no statistically significant change, confirming that the optimizations target only the orchestration layer and do not introduce overhead in untargeted phases. The combined effect is a 1.67× end-to-end median speedup with an average saving of 22.7 seconds per query (40% reduction).

Per-query gains correlate with parallelism. The optimized pipeline achieves greater than 1.0× on 16 of 18 queries, with the largest gains on plans that have multiple independent branches (Q16: 5.06×, Q3: 3.27×, Q6: 3.03×). Two queries show modest regression (Q1: 0.92×, Q11: 0.67×); both regressions trace to LLM-side variance in summarization rather than overhead from the optimizations. Appendix C provides the full per-query distribution and a worked structural comparison for Q6.

4.3 End-to-End Combined Pipeline

We now evaluate the full pipeline (cache + MCP optimizations) versus the unoptimized baseline on 80 paraphrase-tier queries derived from the 20 AOB IoT seed scenarios.

Latency. The baseline achieves a median end-to-end latency of 34.10s (mean 68.68s, range 6.73s to 398.73s). The fully optimized pipeline reduces this to 9.80s (mean 33.06s, range 0.26s to 230.78s), an overall $3.48\times$ median speedup. The cache hit rate is 45.0% (36 of 80 rows). On hit rows the optimized pipeline bypasses plan-execute entirely and returns a cached response, yielding $31.87\times$ median speedup and saving a median of 25.50s per row.

Miss path is still faster. On the 44 miss rows the optimized pipeline still beats the baseline: the median latency difference is -3.30 s. This saving comes from the MCP layer alone. The miss path therefore incurs no net overhead relative to the unoptimized baseline; the cache lookup cost is more than recovered by MCP-side gains. Figure 4 visualizes this: hit rows collapse to near-zero optimized latency, miss rows track below the baseline.

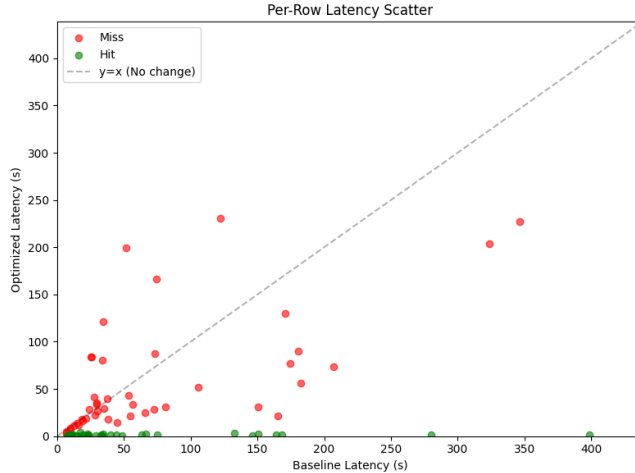


Figure 4: Per-row latency for all 80 evaluation queries. Cache hits collapse to near-zero optimized latency; misses track below the baseline because MCP optimizations apply regardless of cache state.

Cache decision quality and the parameter-sensitivity ceiling. On the combined pipeline the cache reaches precision 0.75, recall 0.5625, F1 0.6429, and specificity 0.7188. Compared with the cache-only configuration in Appendix B, precision improves (0.667 to 0.75) while recall drops slightly, reflecting a more conservative judger when the full pipeline is available as fallback. The residual errors concentrate on parameter-shifted queries: paraphrases that differ only in asset ID or sensor name embed close to seed entries, pass the similarity gate, and require the judger to make a fine-grained distinction that the embedding does not surface. This empirically caps F1 in our setting and motivates parameter-aware judging as future work.

The two layers are additive. The MCP optimizations provide a consistent latency reduction on every query regardless of cache state, while temporal semantic caching adds a large further reduction on the subset of queries that resolve to valid hits. Neither layer undermines the other: the MCP-optimized miss path is faster than the unoptimized baseline, so the cache never makes miss-path performance worse.

5 Limitations and Failure Modes

The most important limit of our approach is structural rather than incidental: pure semantic similarity is not a sound proxy for answer validity in parameter-rich industrial queries, and no choice of similarity threshold can fully resolve this. We discuss this and other limits below.

Parameter-collision false positives. The dominant failure mode is cross-parameter false positives. Queries that share linguistic frame but differ in asset, sensor, or time window can embed at cosine similarity above 0.95, then pass the reranker-based judger above the strict $\tau_{\text{sm}} = 0.92$ threshold, and return an answer drawn from a different operational context. Concretely: in our evaluation we observed a query asking for “Tonnage sensor readings for Chiller 6 *and* Chiller 9 at MAIN site during the first week of *December 2020*” return a cached answer originally produced for “% Loaded data for Chiller 6 at MAIN site for *June 2020*,” with a judger score of 0.97. The two queries differ in three operational dimensions (sensor, equipment scope, time window), but the embedding is dominated by their shared frame (“sensor readings for Chiller X at MAIN site”). This pattern caps hit-decision F1 near 0.64 in the combined system (Section 4.3) and accounts for the 0.5625 recall: tightening τ_{sm} further would suppress these false positives but at the cost of additional missed legitimate paraphrases. Within pure semantic caching, this is unfixable; Section 6 discusses parameter-aware extensions.

Judger inconsistency on legitimate paraphrases. A separate failure mode is judger noise on clear paraphrases. We observed paraphrases of the same seed query receiving judger scores from 0.5 to 0.95 on semantically equivalent content, with no obvious pattern in the residual error. We attribute this to the bounded capacity of Qwen3-Reranker-0.6B, which is the smallest variant of its family. Larger or domain-adapted rerankers may reduce this variance.

Workload structure caps the achievable hit rate. The 80-row paraphrase tier in Section 4.3 is biased toward warm-parent paraphrases by construction (60%/40% warm/cold split), giving a hit rate of 45%. On the full unfiltered AOB corpus the achievable hit rate is structurally lower, between roughly 15% and 30% in our setup, because most AOB queries are parameter-rich data fetches (specific asset IDs, specific sensors, specific time windows) where pure semantic matching is fundamentally unsafe. The setting in which our cache contributes most cleanly is knowledge-style queries (failure-mode enumeration, sensor-to-component mappings, model-support questions), which are a subset of the AOB workload.

Excluded queries and provider variance. Two of the 20 IoT queries used for the MCP workflow experiment (Q5 and Q19) timed out across all attempts in both baseline and optimized configurations and are excluded from Table 1 and Figure 7. These timeouts are upstream LLM failures rather than orchestration failures, but they are worth flagging as a scope limit on the per-query analysis. Additionally, the two MCP regressions in Figure 7 (Q1 at $0.92\times$, Q11 at $0.67\times$) trace to summarization-phase variance under WatsonX provider load: Q11’s summarization took 18.6s in one optimized run versus 7.9s in baseline. This is a property of the LLM endpoint rather than our optimizations, but it does mean per-query speedups have a noise floor we cannot eliminate.

Window grammar and natural-date handling. The temporal classifier resolves a fixed grammar of relative phrases (“yesterday,” “last week,” explicit ISO ranges) into concrete windows. Natural-date phrases like “June 2020” or “the last week of December 2020” currently extract as bucket=Anchored but window=None, so they are demoted to the Static path at lookup. This works correctly but loses the temporal-prefilter benefit for a non-trivial fraction of AOB queries. A richer date parser would promote these into proper Anchored buckets.

Single-machine evaluation, no persistence. All experiments run on a single Apple M-series machine with 16 GB unified memory. Inter-machine variability and concurrent-load effects are out of scope. The cache also lives only in memory: a process restart loses all warmed entries, and there is no checkpoint or replay mechanism. Both are addressed in future work.

6 Future Work

The limitations above point to several concrete extensions, ordered roughly from most committed to most exploratory.

Parameter-aware caching. The natural follow-up to the parameter-collision finding is to extract structured parameters (entity, sensor, time window, action verb) from each query and cache under a key of the form (canonical_intent, param_combo). A parameter-exact lookup short-circuits to the

cached answer; semantic matching only fires when parameter sets overlap. This combines hash-keyed precision with paraphrase-robustness and would eliminate the false-positive class observed in Section 5.

Hybrid retrieval at the lookup layer. A lightweight parameter-extraction layer sits in front of the existing temporal-and-semantic cache. The classifier outputs both a temporal bucket (as today) and a parameter signature; lookup first attempts an exact-parameter match, then falls back to semantic retrieval restricted to entries with overlapping parameters. This integrates naturally with the existing Asteria substrate.

A larger or domain-adapted reranker. Replacing Qwen3-Reranker-0.6B with the 4B variant or with a reranker fine-tuned on AOB-style query-answer pairs may reduce the judger inconsistency described in Section 5.

Richer natural-date grammar. A date parser that handles “June 2020,” “Sept 19 2020 at 7pm,” and similar natural expressions would promote currently-demoted Anchored queries back into the temporal pre-filter path, improving cache hit safety on time-bounded queries.

Cache persistence. Pickling the cache state plus a FAISS index round-trip with versioning would avoid the 30-second pre-warm cost on every process restart. This is mechanical engineering work but is necessary for any non-trivial deployment.

Online judger threshold recalibration. Asteria (18) specifies an online ground-truth-sampling procedure for τ_{sm} tuning. Our implementation currently uses a single offline-set threshold; an online recalibration loop would adapt to workload drift.

Scaling the evaluation. The 152-utterance AOB corpus is small. Generating 1000+ utterances using the existing paraphrase generator, with stratification across query types and parameter shifts, would tighten confidence in the failure-mode and speedup claims.

Integration with serving infrastructure. The temporal cache and MCP optimizations are orthogonal to engine-level optimizations such as PagedAttention (19) or SGLang’s structured-program execution (20). Combining the two layers in a production deployment is a separate engineering exercise but should compound the gains reported here.

7 Conclusion

We presented two complementary optimization layers for AOB plan-execute pipelines: a temporal semantic cache that classifies queries by time-sensitivity before semantic retrieval, and a set of MCP workflow optimizations that eliminate per-query discovery overhead and parallelize independent plan steps. The two layers are additive: MCP optimizations reduce latency on every query regardless of cache state, while temporal semantic caching adds large further savings on the subset of queries that resolve to valid hits. On 18 IoT queries the MCP layer alone yields a $1.67\times$ end-to-end speedup; on the 80-row paraphrase tier the combined pipeline yields $3.48\times$. The miss path is faster than baseline, so the cache never makes performance worse.

Beyond the speedup, the contribution we want reviewers to weigh is the failure-mode analysis. Pure semantic similarity, even paired with a strict reranker-based judger, is not a sound proxy for answer validity in parameter-rich industrial queries: shared linguistic frame dominates the embedding, distinct operational parameters do not. Our hit-decision F1 caps near 0.64 in this setting, with the residual error concentrated on cross-parameter false positives that pass even at $\tau_{\text{sm}} = 0.92$. This is a structural property of pure semantic caching as an evaluation/optimization choice for MCP-backed industrial agents, not a tuning issue, and it gives a concrete handle on when caching is safe to deploy as part of an evaluation pipeline. Parameter-aware caching, the natural next step, is laid out in Section 6.

Acknowledgments

We thank Dhaval Patel and the IBM team for providing access to AssetOpsBench and for their guidance throughout this work. We thank Kaoutar El Maghraoui for her instruction and mentorship over the course of this project.

References

- [1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [2] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [3] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive APIs. *Advances in Neural Information Processing Systems (NeurIPS)*, 37:126544–126565, 2024.
- [4] Anthropic. Model Context Protocol (MCP) specification. <https://modelcontextprotocol.io>, 2024.
- [5] Dhaval Patel, Shuxin Lin, James Rayfield, Nianjun Zhou, Roman Vaculin, Natalia Martinez, Fearghal O’donncha, and Jayant Kalagnanam. AssetOpsBench: Benchmarking AI agents for task automation in industrial asset operations and maintenance, 2025.
- [6] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 6, pages 325–338, 2024.
- [7] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, pages 94–109, 2025.
- [8] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. RAGCache: Efficient knowledge caching for retrieval-augmented generation. *arXiv preprint arXiv:2404.12457*, 2024.
- [9] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. CacheGen: KV cache compression and streaming for fast large language model serving. *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 38–56, 2024.
- [10] Fu Bang. GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS)*, pages 212–218, 2023.
- [11] Luis Gaspar Schroeder, Shu Liu, Alejandro Cuadron, Mark Zhao, Stephan Krusche, Alfons Kemper, Matei Zaharia, and Joseph E Gonzalez. Adaptive semantic prompt caching with VectorQ. *arXiv preprint arXiv:2502.03771*, 2025.
- [12] Qizheng Zhang, Michael Wornow, Gerry Wan, and Kunle Olukotun. Agentic plan caching: Test-time memory for fast and cost-efficient LLM agents. *arXiv preprint arXiv:2506.14852*, 2025. NeurIPS 2025.
- [13] Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. In *International Conference on Machine Learning (ICML)*, 2024.
- [14] Charles Packer, Vivian Fang, Shishir G Patil, Kevin Lin, Sarah Wooders, and Joseph E Gonzalez. MemGPT: Towards LLMs as operating systems, 2023.

- [15] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.
- [16] Theodore Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2023.
- [17] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2024.
- [18] Chaoyi Ruan, Chao Bi, Kaiwen Zheng, Ziji Shi, Xinyi Wan, and Jialin Li. Asteria: Semantic-aware cross-region caching for agentic LLM tool access. *arXiv preprint arXiv:2509.17360*, 2025.
- [19] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023.
- [20] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, and Joseph E Gonzalez. SGLang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems (NeurIPS)*, 37:62557–62583, 2024.
- [21] Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-agents enhances large language model capabilities. *arXiv preprint arXiv:2406.04692*, 2024.
- [22] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024.
- [23] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, pages 1–22, 2023.
- [24] Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general AI assistants. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [25] Avanika Narayan, Dan Biderman, Sabri Eyuboglu, Avner May, Scott Linderman, James Zou, and Christopher Ré. Minions: Cost-efficient collaboration between on-device and cloud language models. *arXiv preprint arXiv:2502.15964*, 2025.

A Implementation Parameters

Discovery cache. The cache key is computed as an MD5 hash over three components: the registered server paths, the last-modified timestamps (`mtime`) of all Python source files within the `src/servers/` directories, and the modification time of the repository’s `pyproject.toml` dependency file. Any change to server logic or project dependencies invalidates the key automatically. We additionally enforce a 24-hour time-to-live (TTL).

Parallel executor. The executor uses Kahn’s algorithm to group plan steps into topological dependency layers and dispatches all steps within a layer concurrently using `asyncio.gather()`. The `MCPServerPool` maintains one persistent `stdio` session per required server, with per-server asynchronous locks serializing concurrent calls to the same server.

Semantic cache models and thresholds. Embeddings are produced by `Qwen3-Embedding-0.6B` (1024-dim). The judge is `Qwen3-Reranker-0.6B` run with prefill-only inference. Both models run on Apple Silicon MPS in `fp16`. ANN retrieval uses `FAISS` with `top_k=5` and a coarse cosine threshold of `tau_sim=0.75`. The judge applies a strict acceptance threshold of `tau_jsm=0.92`. Cache capacity is 50 entries with LCFU eviction.

LLM backend. Planning, tool-argument resolution, and summarization use `watsonx/meta-llama/llama-3-3-70b-instruct` via `LiteLLM`.

Hardware. All experiments run on a single Apple M-series machine with 16 GB unified memory.

B Cache-Only Configuration: Decision Breakdown

We additionally evaluate the temporal semantic cache as a standalone layer on top of the unmodified plan-execute pipeline (no MCP optimizations) on a stratified sample of 50 queries. The cache achieves a hit rate of 36.0%, with median speedup $30.62\times$ on hits and median overhead $+2.23s$ on misses. Decision quality reaches precision 0.667, recall 0.667, F1 0.667, specificity 0.813. Figure 5 shows boxplot for asteria optimization; Figure 6 shows per-row latency.

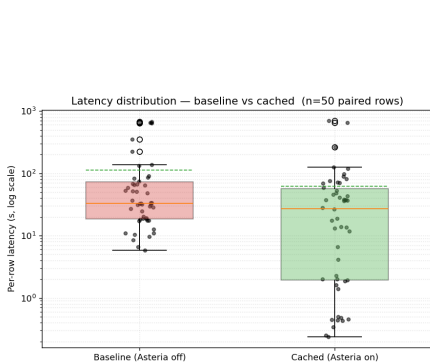


Figure 5: Box plot of baseline and cached latency distributions across the 50 evaluation rows.

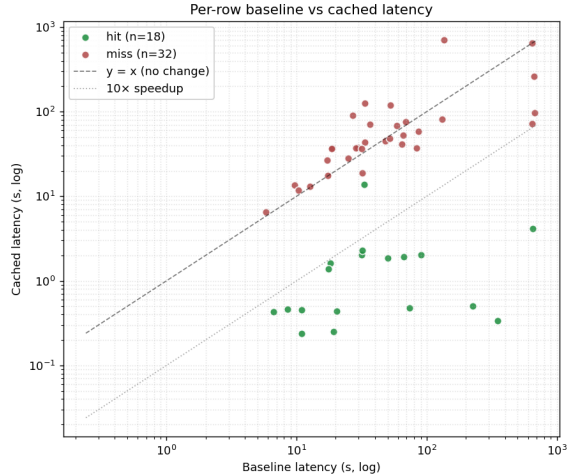


Figure 6: Cache-only per-row latency. Hits collapse to near-zero cached cost regardless of baseline.

C Per-Query Speedup and Structural Comparison

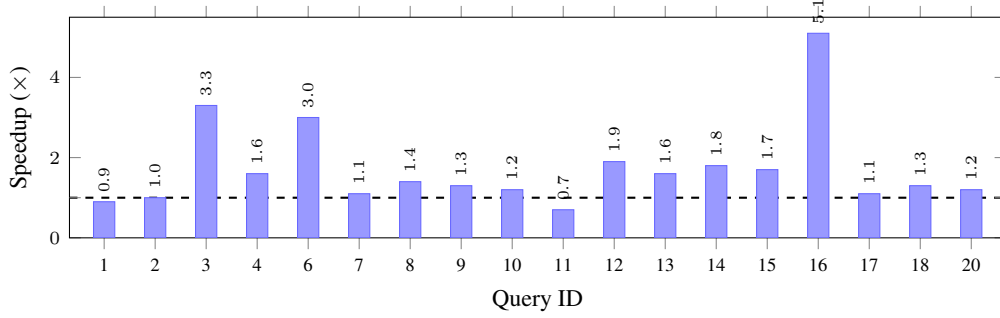
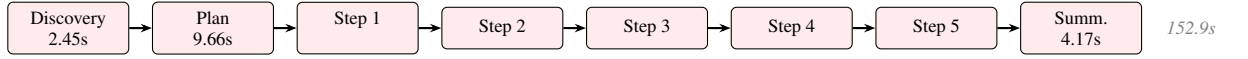


Figure 7: Per-query end-to-end speedup across 18 completed IoT queries. Dashed line marks 1.0 $\times$ break-even.

Figure 8 shows a worked structural comparison for Query 6 (failure modes of Chiller 6 detectable by its Chiller Efficiency sensor; 5-step plan, two dependency layers). In the baseline path each step spawns a fresh MCP subprocess, executes a single tool call, and terminates before the next step begins. In the optimized path the server pool starts each required server once, the discovery cache bypasses spawning entirely, and independent steps within each dependency layer execute concurrently. Total latency drops from 152.9s to 50.5s, a 3.03 $\times$ speedup.

Baseline (Sequential)



Optimized (Parallel + Cache + Pool)

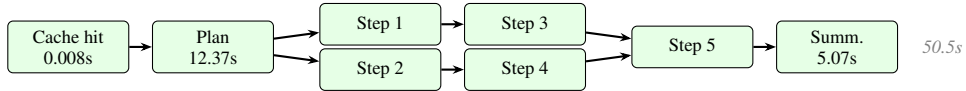


Figure 8: Workflow comparison for Q6. Top: baseline sequential execution with subprocess-per-call. Bottom: optimized execution with discovery cache, parallel DAG layers, and persistent server pool.

D Broader Impact and Societal Implications

Positive impacts. Reducing per-query latency and API cost makes LLM-backed agent systems more accessible to organizations that cannot afford high-throughput commercial serving. In industrial operations, faster and cheaper query resolution can improve response times for maintenance workflows, equipment fault detection, and work-order management, with downstream benefits for operational safety and efficiency. The workflow optimizations are backend-agnostic and can compound with engine-level improvements such as PagedAttention, so their benefits extend beyond the specific AOB setting studied here.

Potential negative impacts. The primary risk introduced by any caching layer is stale or incorrect answer reuse. In safety-critical industrial settings (e.g., returning a cached sensor reading that no longer reflects current equipment state), an incorrect cache hit could inform a faulty maintenance decision. Our temporal classifier mitigates this by routing live-state queries past the cache, but it does not eliminate the risk entirely, particularly for Anchored queries whose window grammar does not parse correctly. Practitioners deploying this system in safety-critical contexts should treat cache hits as advisory and maintain a fallback to live query execution. No direct path to misuse for disinformation, surveillance, or discriminatory decision-making is introduced by this work.